

Quest1 - Problem statement and solution

Given a video link and a line of dialogue, return the exact moment that line is spoken, and hands you a picture of the screen at that moment.

Core Idea and Architecture Diagram

A video is two things playing side by side on a single shared clock: a soundtrack and a strip of still pictures. Because they share that clock, anything you learn about the sound is automatically true of the picture. If a computer listens to the soundtrack and works out that the word “trunks” begins at second 7.38, then second 7.38 of the picture is exactly the frame you want.

So the model never searches the pictures at all - that would be enormously expensive. It searches the words, gets back a time, and then uses that time to fetch a single frame. This is also why it never downloads the video: it only needs the sound, plus a few hundred kilobytes for the one still it ends up showing you.

That principle splits the work in two. Steps 1-4 below understand the video and have nothing to do with the question being asked; steps 5-6 are the only parts that depend on the line actually typed. Everything the first four produce is written into a folder named after the video, so a second search on it skips straight to matching and answers in about a second rather than minutes. Only the very first question asked of a video pays for the recognition; every question after it is close to free.

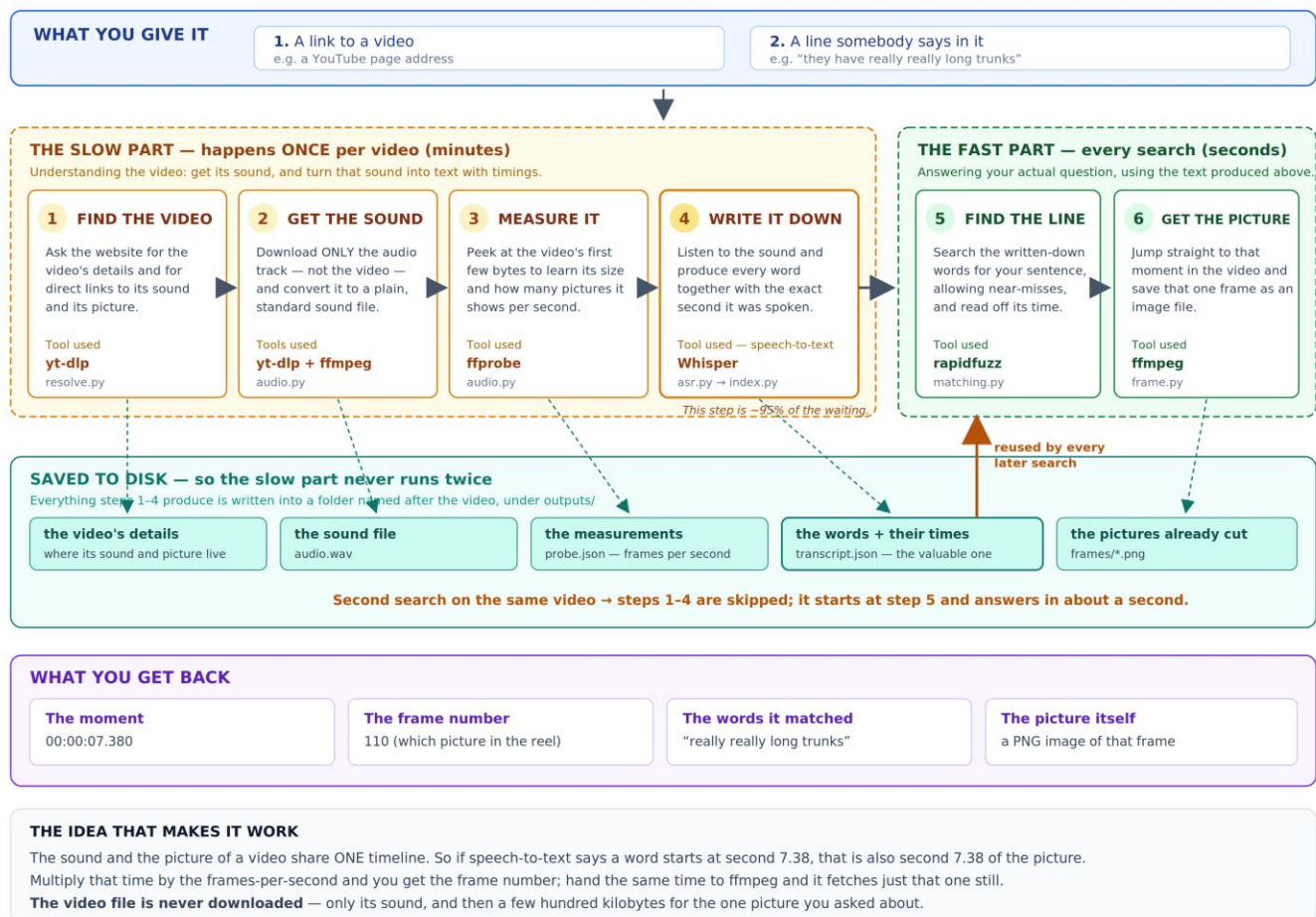


Figure 1. Architecture diagram.

Workflow

- 1. Find the video.** (`resolve.py` · `yt-dlp`) A video page doesn't contain the video, only instructions for fetching it. `yt-dlp` reads those and returns the title, the duration and a list of separately downloadable streams, from which one audio and one video stream are chosen. The video's identity is hashed from the site name and video ID - not from the stream address, which expires within hours, nor the title, which can be edited - and that hash becomes the folder everything is stored under. Nothing is downloaded yet.
- 2. Get the sound.** (`audio.py` · `yt-dlp` + `ffmpeg`) Only the audio stream is fetched, pulled in parallel byte-range chunks rather than one long read: hosts throttle a single continuous read, and this change alone took one test video's audio stage from 399 seconds to 8.7. `ffmpeg` transcodes it to 16 kHz mono PCM. The result's length is checked against the known duration, and a shortfall is rejected as a truncated download - half a soundtrack would confidently report "not found" for the missing half.
- 3. Measure the video.** (`audio.py` · `ffprobe`) `ffprobe` requests only the video stream's container header - the label on the tin, without opening it — and reads back the width, height and the two frame-rate fields the file declares. If those two disagree by more than 1% the frame rate varies across the file, and no frame number can be derived by arithmetic. The measurements are stored beside the video's permanent page address, since the stream address will expire.
- 4. Write down every word.** (`asr.py` → `index.py` · `Whisper`) `Whisper` transcribes the audio with word-level timestamps, so each word carries the second it starts and ends; silence-skipping both speeds up decoding and stops the model inventing words during quiet stretches. The transcript is then reshaped into one long normalised string of every word, plus an array recording which word each character belongs to - the bridge between the search library, which reports character positions, and the timings, which are held per word. This step is about 95% of the wait, and the reason everything is written to disk.
- 5. Find your line.** (`matching.py` · `rapidfuzz`) The query is folded through the same normalisation used when the index was built, so casing and punctuation cannot cause a miss. `rapidfuzz` scores it against that string; the cut-off of 70 out of 100 was set by testing real phrases taken from other videos, which produced no false matches at 70 and several at 65. The best span is then blanked out and the search repeated so every occurrence is found, and each span is mapped back through the array to word indices - the first word's start time is the answer.
- 6. Get the picture.** (`frame.py` · `ffmpeg`) The frame number is the time multiplied by frames-per-second, rounded down, since the frame on screen at an instant is the one whose slice of time contains it. The image comes from a two-stage seek: a coarse jump lands on a keyframe using byte ranges alone, then a short decode forward stops on the exact frame - so only a few hundred kilobytes cross the network. An expired stream address is refreshed and retried once, and the saved file is verified to be a real image.

All six run inside a single function; the terminal tool and the web server are thin wrappers around it, the web layer running it on a background thread and streaming progress to the browser so a long search shows a moving bar rather than a frozen page.

Significance

- **The expensive work is paid for once, not once per question.** Understanding a video and answering a question about it are separated cleanly, so a search costing minutes the first time costs about a second every time after. Caching here is not an optimisation added at the end - it is the shape of the design.
- **It answers a question about video without ever handling video.** Moving the search into the transcript and using the shared timeline to come back out again solves an expensive visual problem with cheap text operations; a 90-minute film leaves roughly half a megabyte on disk.
- **It refuses to guess.** Where a frame number cannot honestly be derived it returns nothing and explains why; where a line is said more than once it returns every occurrence instead of silently choosing; where nothing matches it shows the closest lines in the transcript. The thresholds behind those decisions come from recorded experiments, not assumption.
- **One engine, two front ends, stages that stand alone.** Every stage is a separate module with one job, runnable by itself from the terminal, and both front ends call the same function with the same validation - so faults can be traced to a single stage and the two interfaces cannot drift apart.